# Vision Document: CalcEngine Scientific Calculator API

**Author:** Raja SP, AWS Labs
**Source:** https://github.com/awslabs/aidlc-workflows
**Adaptation:** Ricardo de Luna Galdino, EngSoft Learn
**Download:** example-vision-scientific-calculator-api.pdf

## Executive Summary

CalcEngine is a hosted scientific calculator library delivered as a REST API that enables software development teams to embed accurate, standards-compliant mathematical computation into their own applications without building or maintaining calculation logic themselves. It addresses the recurring problem of engineering teams spending months implementing, testing, and debugging mathematical functions that are peripheral to their core product. The expected outcome is a subscription API service generating $2M ARR within 24 months by capturing developers building in education technology, financial modeling, engineering simulation, and data analysis.

---

## Business Context

### Problem Statement

Software teams building products in education, finance, engineering, and science regularly need mathematical computation beyond basic arithmetic. They face a choice: use a local library (often language-specific, inconsistent across platforms, and a maintenance burden) or build the math functions themselves (expensive, error-prone, and slow).

The specific problems are:

- **Accuracy risk**: Teams without mathematics expertise introduce subtle floating-point errors, incorrect edge-case handling (division by zero, overflow, domain errors), and

inconsistent rounding behavior that erode trust in their products.

- **Duplicated effort**: Every team that needs trigonometric functions, statistical distributions, matrix operations, or unit conversions builds them independently. This work is repeated across thousands of companies.
- **Cross-platform inconsistency**: A calculation performed in a Python backend may produce a different result than the same calculation in a JavaScript frontend. Customers who operate across platforms cannot guarantee consistency.
- **Compliance and auditability**: In regulated industries (finance, healthcare, engineering), calculations must be traceable, versioned, and validated. Ad-hoc implementations rarely meet audit requirements.

## Business Drivers

- **API-first economy**: Developers increasingly prefer consuming hosted APIs over embedding libraries. Stripe (payments), Twilio (communications), and SendGrid (email) have proven the model. No equivalent exists for mathematical computation.
- **EdTech growth**: The global education technology market is expanding rapidly, and calculator functionality is a universal requirement across math, science, and engineering courseware.
- **Regulatory pressure**: Financial services firms face increasing scrutiny on calculation accuracy. A certified, auditable API reduces their compliance burden.
- **AI/ML preprocessing**: Data science teams need reliable mathematical transformations as preprocessing steps. An API that guarantees precision and reproducibility has clear value in ML pipelines.

## Target Users and Stakeholders

| User Type | Description | Primary Need |
|---|---|---|
| Application Developer | Backend or frontend engineer integrating math into a product | Reliable, well-documented API with consistent results across calls |
| EdTech Product Manager | Non-technical stakeholder at an education company | Confidence that calculation results shown to students are correct |
| Financial Analyst / Quant Developer | Developer building financial models or trading systems | Arbitrary-precision arithmetic with auditable, versioned calculation history |
| Engineering Simulation Developer | Engineer building CAD, physics, or modeling tools | High-performance matrix, vector, and differential equation operations |
| Data Scientist | Analyst building preprocessing pipelines | Consistent statistical functions callable from any language or platform |
| DevOps / Platform Engineer | Engineer responsible for uptime and integration | Low-latency, high-availability API with clear SLAs and monitoring |

## Business Constraints

- **Bootstrap budget**: Initial development funded from existing revenue. No external investment planned for MVP. Total MVP budget is $150K (covering development, infrastructure, and initial marketing).
- **Small team**: Two backend developers, one frontend developer (for documentation portal), one QA engineer. No dedicated math PhD on staff for MVP phase.
- **Time to market**: MVP must be publicly available within 6 months to validate demand before committing to Phase 2 investment.
- **Pricing model**: Must support a free tier (to drive adoption) and usage-based paid tiers. Pricing infrastructure needed at launch.

- **Accuracy standards**: All functions must match or exceed the accuracy of IEEE 754 double-precision arithmetic. Arbitrary-precision mode is a Phase 2 feature, not MVP.

## Success Metrics

| Metric | Current State | Target State (12 months post-MVP) | Measurement Method |
|---|---|---|---|
| Registered API accounts | 0 | 5,000 | API key registration count |
| Monthly active API consumers | 0 | 1,200 | Unique API keys making at least 1 call/month |
| API calls per month | 0 | 10 million | CloudWatch API Gateway metrics |
| Paid subscribers | 0 | 200 | Billing system records |
| Monthly recurring revenue | $0 | $80K | Billing system records |
| API uptime | N/A | 99.9% | CloudWatch availability monitoring |
| Mean response time (p50) | N/A | < 50ms | CloudWatch latency metrics |
| Customer-reported accuracy bugs | N/A | < 5 per quarter | Support ticket tracking |
| Developer documentation satisfaction | N/A | > 4.2 / 5.0 | Quarterly survey of registered developers |

# Full Scope Vision

## Product Vision Statement

CalcEngine becomes the default computation layer for any application that needs mathematical operations beyond basic arithmetic, the way Stripe became the default for payments, by offering an API that is more accurate, more consistent, and easier to integrate than building it yourself.

## Feature Areas

### Feature Area 1: Core Arithmetic and Algebra

- **Description**: Fundamental mathematical operations that go beyond what standard language math libraries provide reliably.
- **Key Capabilities**:
- Arbitrary-precision integer and decimal arithmetic (configurable precision up to 1,000 digits)
- Expression parsing and evaluation (accept string expressions like "2 * sin(pi/4) + log(100)")
- Polynomial operations (evaluation, root finding, factoring)
- Equation solving (linear, quadratic, cubic, systems of linear equations)
- Fraction and rational number arithmetic (exact representation, simplification)
- Complex number arithmetic (addition, multiplication, polar/rectangular conversion)
- **User Value**: Developers send a math expression as a string and get a precise, verified result without implementing a parser or worrying about operator precedence, floating-point drift, or edge cases.

### Feature Area 2: Trigonometry and Geometry

- **Description**: Complete trigonometric and geometric calculation capabilities.
- **Key Capabilities**:
- All six trigonometric functions and their inverses (sin, cos, tan, csc, sec, cot)
- Hyperbolic functions and inverses
- Degree/radian/gradian conversion
- Coordinate system conversions (Cartesian, polar, spherical, cylindrical)
- Geometric calculations (area, volume, perimeter for standard shapes)
- Distance and angle calculations in 2D and 3D space

- **User Value**: Eliminates the need to implement trigonometric edge cases (exact values at special angles, quadrant handling, domain validation).

**Feature Area 3: Statistics and Probability**

- **Description**: Statistical analysis and probability distribution functions.
- **Key Capabilities**:
- Descriptive statistics (mean, median, mode, variance, standard deviation, quartiles, percentiles)
- Probability distributions (normal, binomial, Poisson, chi-squared, t-distribution, F-distribution) with PDF, CDF, and inverse CDF
- Regression analysis (linear, polynomial, exponential, logarithmic)
- Hypothesis testing (t-test, chi-squared test, ANOVA)
- Combinatorics (permutations, combinations, factorial, binomial coefficients)
- Random number generation with configurable distributions and seeds
- **User Value**: A single API call replaces importing and configuring statistical libraries. Results are reproducible and auditable.

**Feature Area 4: Linear Algebra and Matrix Operations**

- **Description**: Matrix and vector computation for engineering, graphics, and data science.
- **Key Capabilities**:
- Matrix arithmetic (addition, multiplication, scalar operations)
- Matrix decompositions (LU, QR, SVD, Cholesky, eigenvalue)
- Determinant, inverse, rank, trace
- Vector operations (dot product, cross product, normalization)
- Systems of linear equations (Gaussian elimination, least squares)
- Sparse matrix support for large-scale problems
- **User Value**: Teams building simulation, ML, or graphics applications get validated linear algebra without linking to LAPACK or maintaining native bindings.

**Feature Area 5: Calculus**

- **Description**: Symbolic and numerical calculus operations.
- **Key Capabilities**:
- Numerical differentiation (first and higher-order derivatives)
- Numerical integration (definite integrals with configurable methods: trapezoidal, Simpson's, Gaussian quadrature)

- Symbolic differentiation and integration (for supported expression types)
- Limits and series expansion (Taylor, Maclaurin)
- Ordinary differential equation solvers (Euler, Runge-Kutta)
- **User Value**: Engineers and scientists get calculus operations via API without embedding a computer algebra system.

**Feature Area 6: Unit Conversion and Physical Constants**

- **Description**: Standard unit conversion and access to verified physical and mathematical constants.
- **Key Capabilities**:
- Unit conversion across all SI and common imperial units (length, mass, temperature, time, energy, pressure, speed, etc.)
- Currency conversion (with daily rate updates from a financial data provider)
- Physical constants (speed of light, Planck's constant, Avogadro's number, etc.) with cited sources and uncertainty values
- Mathematical constants to configurable precision (pi, e, golden ratio, etc.)
- Dimensional analysis (validate that unit combinations are physically meaningful)
- **User Value**: One API replaces multiple conversion libraries and hardcoded constant values, with the guarantee that constants are sourced and current.

**Feature Area 7: Financial Mathematics**

- **Description**: Financial calculation functions for lending, investment, and risk analysis.
- **Key Capabilities**:
- Time value of money (present value, future value, annuities, perpetuities)
- Loan amortization schedules
- Bond pricing and yield calculations
- Option pricing (Black-Scholes, binomial model)
- Internal rate of return (IRR) and net present value (NPV)
- Depreciation methods (straight-line, declining balance, sum-of-years)
- **User Value**: FinTech companies get auditable, regulation-ready financial calculations without building proprietary math engines.

**Feature Area 8: Developer Experience and Platform**

- **Description**: The API platform, documentation, SDKs, and developer tools that make CalcEngine easy to adopt.

- **Key Capabilities**:
- Interactive API documentation with live "try it" sandbox
- Client SDKs for Python, JavaScript/TypeScript, Java, C#, Go, and Ruby
- Webhook support for long-running calculations (batch processing)
- Calculation history and audit log per API key
- Rate limiting with clear quotas and overage handling
- API versioning with 12-month deprecation policy
- Workspace feature for teams (shared API keys, usage dashboards, billing management)
- **User Value**: Developers can go from signup to first successful API call in under 5 minutes.

## Integration Points

- **Payment processor** (Stripe) - Subscription billing and usage-based metering
- **Identity provider** (Auth0 or Cognito) - Developer account authentication
- **Financial data provider** (for currency rates) - Daily exchange rate feeds
- **NIST / CODATA** - Source of truth for physical constants
- **CI/CD systems** (GitHub Actions, GitLab CI) - SDK publishing and version management
- **Monitoring** (Datadog or CloudWatch) - API performance, error rates, usage dashboards

## User Journeys (Full Vision)

### Journey 1: EdTech Developer Adds Calculation to a Course Platform

1. Developer discovers CalcEngine through a search for "scientific calculator API" and lands on the documentation site.
2. Developer creates a free account and gets an API key in under 2 minutes.
3. Developer browses the interactive documentation, finds the trigonometry endpoint, and tests `sin(pi/4)` in the sandbox.
4. Developer installs the Python SDK via pip and writes a 3-line integration that sends student-entered expressions to CalcEngine and displays the result.
5. Developer configures the API to return step-by-step solution breakdowns so students can see how the answer was derived.
6. Course platform goes live. Thousands of students submit calculations daily. The developer monitors usage through the CalcEngine dashboard and upgrades to a

paid tier when free-tier limits are reached.

**Outcome**: The education platform ships a reliable calculator feature in one afternoon instead of spending weeks building and testing math parsing.

**Journey 2: FinTech Startup Builds a Loan Comparison Tool**

1. Product team at a lending startup needs amortization schedules, APR calculations, and present-value computations for a customer-facing loan comparison tool.
2. Developer signs up for CalcEngine and navigates to the Financial Mathematics section.
3. Developer uses the loan amortization endpoint to generate a payment schedule for a 30-year mortgage at 6.5% interest. The API returns month-by-month principal, interest, and balance breakdowns.
4. Developer integrates the NPV and IRR endpoints to let customers compare different loan offers side by side.
5. Compliance team reviews CalcEngine's accuracy certification and audit log. Each calculation is traceable to a versioned API call with timestamped inputs and outputs.
6. The loan comparison tool launches. CalcEngine handles 500K calculations per month. The startup pays based on usage and avoids hiring a quant developer.

**Outcome**: The FinTech startup launches a compliant, auditable financial tool without building proprietary calculation logic.

**Journey 3: Data Scientist Uses CalcEngine in an ML Pipeline**

1. A data scientist at a healthcare company needs to normalize patient measurement data using statistical transformations (z-scores, percentile ranks, log transforms) as preprocessing before model training.
2. Data scientist installs the CalcEngine Python SDK and calls the statistics endpoints from within a Jupyter notebook.
3. The SDK accepts arrays of values and returns descriptive statistics and transformed datasets.
4. The data scientist configures batch mode to process 100K records. CalcEngine returns results via webhook when processing completes.
5. Because CalcEngine guarantees reproducible results (same inputs, same outputs, across versions), the scientist can cite the API version in their research paper for reproducibility.

**Outcome**: The scientist gets validated, reproducible statistical transformations without writing and debugging custom statistics code.

## Scalability and Growth

- **Geographic expansion**: Initial deployment in US-East. Expand to EU-West and AP-Southeast within 12 months of MVP based on user geography data.
- **Volume growth**: Architect for 1 billion API calls/month within 3 years. Start serverless, migrate high-traffic endpoints to containers if latency requires it.
- **Feature growth**: New feature areas added based on customer demand data. Candidates include: number theory, graph theory, signal processing, optimization solvers.
- **Enterprise expansion**: Introduce on-premises deployment option for regulated industries that cannot send data to a shared API. Target Phase 3.
- **Marketplace presence**: List on AWS Marketplace, Azure Marketplace, and RapidAPI for additional distribution channels.

## Long-Term Roadmap

| Phase | Focus | Timeframe |
|---|---|---|
| MVP | Core arithmetic, trigonometry, basic statistics, expression evaluation, API platform, documentation portal, free + paid tiers | Months 1-6 |
| Phase 2 | Linear algebra, calculus, financial math, arbitrary-precision mode, client SDKs (5 languages), calculation audit log, team workspaces | Months 7-14 |
| Phase 3 | Unit conversion, physical constants, step-by-step solutions, batch processing, enterprise features, on-premises option | Months 15-22 |
| Phase 4 | Advanced statistics (hypothesis testing, regression), symbolic computation, optimization solvers, marketplace listings | Months 23-30 |

---

# MVP Scope

## MVP Objective

Prove that developers will pay for a hosted scientific calculator API by launching with core mathematical functions, validating adoption through free-tier signups, and converting at least 50 accounts to paid plans within 6 months of launch.

## MVP Success Criteria

- [ ] 1,000 registered developer accounts within 3 months of launch
- [ ] 300 monthly active API consumers (at least 1 call/month) within 3 months
- [ ] 50 paid subscribers within 6 months
- [ ] $15K MRR within 6 months
- [ ] API uptime of 99.9% over first 3 months
- [ ] Mean response time (p50) under 50ms for all MVP endpoints
- [ ] Zero critical accuracy bugs reported (calculations returning wrong results)
- [ ] Net Promoter Score of 40+ from developer survey at 3-month mark

## Features In Scope (MVP)

| Feature | Description | Priority | Rationale for Inclusion |
|---|---|---|---|
| Basic arithmetic operations | Add, subtract, multiply, divide, modulo, power, square root, nth root, absolute value, floor, ceiling, rounding | Must Have | Foundation for all other calculations. Table stakes for any calculator API. |
| Expression evaluation | Accept a string math expression (e.g., "2 * (3 + 4)^2 / sin(pi)") and return the evaluated result. Support operator precedence, parentheses, and nested functions. | Must Have | The single most valuable differentiator. Developers send expressions as strings instead of building parsers. |
| Trigonometric functions | sin, cos, tan, asin, acos, atan, atan2 with degree and radian mode | Must Have | Universal requirement across EdTech, engineering, and graphics use cases. |
| Logarithmic and exponential functions | log (base 10), ln (natural log), log with arbitrary base, exp, power | Must Have | Required for financial, scientific, and statistical calculations. |
| Basic statistics | Mean, median, mode, standard deviation, variance, min, max, sum, count, percentile | Must Have | High-frequency need. Validates demand from data science and EdTech segments. |
| Mathematical constants | pi, e, golden ratio (phi), sqrt(2), sqrt(3), ln(2), ln(10) to IEEE 754 double precision | Must Have | Low implementation cost, high utility. Prevents developers from hardcoding imprecise values. |

| Feature | Description | Priority | Rationale for Inclusion |
|---|---|---|---|
| Factorial, permutations, combinations | n!, nPr, nCr with large number support | Must Have | Required for probability and combinatorics use cases in EdTech. |
| Error handling and domain validation | Clear error responses for domain errors (sqrt of negative, log of zero, division by zero), overflow, and invalid expressions. Structured error format with error codes. | Must Have | Professional API quality. Bad error handling is the top reason developers abandon APIs. |
| API key management | Developer signup, API key generation, key rotation, key revocation | Must Have | Minimum authentication infrastructure for a commercial API. |
| Usage metering and rate limiting | Track calls per API key. Free tier: 10,000 calls/month. Paid tier: 1M calls/month. Clear rate limit headers in responses. | Must Have | Revenue model depends on usage-based pricing. Must be present at launch. |
| REST API with JSON | All endpoints accept JSON, return JSON. Standard REST conventions. OpenAPI 3.x specification published. | Must Have | Expected standard for modern APIs. |
| API documentation portal | Hosted documentation site with endpoint reference, code examples in 3 languages (Python, JavaScript, cURL), and interactive "try it" sandbox. | Must Have | Developer adoption depends entirely on documentation quality. |
| Billing integration | Stripe-based subscription billing. Free tier, Starter ($29/mo), Professional | Must Have | Revenue collection must be automated from day one. |

| Feature | Description | Priority | Rationale for Inclusion |
|---|---|---|---|
| | ($99/mo). Usage overage billing. | | |

## Features Explicitly Out of Scope (MVP)

| Feature | Reason for Deferral | Target Phase |
|---|---|---|
| Arbitrary-precision arithmetic | Adds significant complexity to every endpoint. Standard IEEE 754 double precision is sufficient for MVP validation. | Phase 2 |
| Matrix and linear algebra operations | Large feature surface area. Not needed to validate core business hypothesis. | Phase 2 |
| Calculus (differentiation, integration) | Requires numerical methods expertise and extensive edge-case testing. | Phase 2 |
| Financial mathematics | Specialized domain. Validate general developer demand first. | Phase 2 |
| Symbolic computation | Requires a computer algebra system. Out of scope for small team and MVP timeline. | Phase 3 |
| Step-by-step solution breakdowns | High value for EdTech but significant implementation effort. Validate demand through customer interviews during MVP. | Phase 3 |
| Unit conversion | Useful but not core to calculator value proposition. Many free alternatives exist. | Phase 3 |
| Physical constants database | Low implementation cost but low urgency. Include in Phase 3 with unit conversion. | Phase 3 |
| Client SDKs (Python, JS, Java, etc.) | Documentation with cURL and code examples is sufficient for MVP. SDKs accelerate adoption but are not required to validate demand. | Phase 2 |
| Batch processing / webhooks | Needed for high-volume users. MVP focuses on synchronous single-calculation calls. | Phase 3 |
| Calculation audit log | Important for regulated industries. Not needed for initial developer adoption. | Phase 2 |

| Feature | Reason for Deferral | Target Phase |
|---|---|---|
| Team workspaces | Enterprise feature. Individual developer accounts are sufficient for MVP. | Phase 3 |
| On-premises deployment | Enterprise feature requiring significant packaging effort. | Phase 3+ |
| Probability distributions (PDF, CDF) | Useful but not core to MVP validation. Basic statistics covers initial demand. | Phase 2 |
| Regression analysis | Specialized statistical feature. Defer until statistics demand is validated. | Phase 4 |
| Complex number arithmetic | Niche use case. Validate demand from engineering users first. | Phase 2 |

## MVP User Journeys

### Journey 1: Developer Discovers and Integrates CalcEngine

1. Developer searches for "math expression evaluation API" and finds CalcEngine documentation.
2. Developer clicks "Get API Key" and completes a one-page signup form (email, password, company name optional).
3. Developer receives API key immediately on the confirmation page and in a welcome email.
4. Developer copies a cURL example from the documentation and runs it in their terminal: `curl -X POST https://api.calcengine.io/v1/evaluate -H "Authorization: Bearer {key}" -d '{"expression": "sin(pi/4) * 2 + sqrt(16)"}'`
5. Developer receives a JSON response: `{"result": 5.414213562373095, "expression": "sin(pi/4) * 2 + sqrt(16)", "precision": "double"}`
6. Developer reads the Python code example on the documentation site, copies it into their application, and replaces the expression string with user input.
7. Application is live. Developer monitors usage on the CalcEngine dashboard.

**Outcome**: Developer goes from discovery to working integration in under 15 minutes.
**Limitation vs Full Vision**: No SDK (raw HTTP calls), no step-by-step breakdowns, no

audit log.

**Journey 2: EdTech Company Evaluates CalcEngine for Student Use**

1. EdTech product manager asks their developer to evaluate CalcEngine for a homework-checking feature.
2. Developer signs up for the free tier and tests 20 common student calculations (quadratic formula, trig identities, basic statistics) using the API sandbox.
3. Developer verifies results against known correct answers. All match.
4. Developer integrates CalcEngine into the homework checker. Students type math expressions, the app sends them to CalcEngine, and the result is compared against the expected answer.
5. Free tier handles initial classroom pilot (500 students, ~8,000 calls/month). When the pilot expands to the full school district, the developer upgrades to the Starter plan.

**Outcome**: EdTech company ships a homework-checking feature without building a math parser.
**Limitation vs Full Vision**: No step-by-step solutions for students, no complex number support, no calculus functions for advanced courses.

## MVP Constraints and Assumptions

- **Assumption**: Developers prefer a hosted API over a local library for math operations. **Risk if wrong**: Low adoption despite accurate computation. **Mitigation**: Free tier allows low-commitment validation; pivot to open-source library model if API model fails.
- **Assumption**: Expression evaluation (string-in, result-out) is the highest-value feature. **Risk if wrong**: Developers actually want individual function endpoints more than expression parsing. **Mitigation**: MVP includes both expression evaluation and individual function endpoints; usage data will reveal which is preferred.
- **Assumption**: IEEE 754 double precision is sufficient for MVP users. **Risk if wrong**: Early adopters in finance or science demand higher precision immediately. **Mitigation**: Arbitrary precision is Phase 2 priority and can be accelerated if demand signals are strong.
- **Assumption**: 10,000 free calls/month is enough to evaluate the product but low enough to drive paid conversion. **Risk if wrong**: Free tier is either too generous (no conversion) or too restrictive (users leave before evaluating). **Mitigation**: Adjust limit based on conversion data at 2-month mark.

- **Accepted Limitation**: No client SDKs at MVP. Developers must make raw HTTP calls. This adds friction but SDKs are expensive to build and maintain across multiple languages before product-market fit is validated.
- **Accepted Limitation**: Single-region deployment (US-East-1). Latency for users in Europe and Asia will be higher. Acceptable for MVP because calculation payloads are small (low bandwidth sensitivity).

## MVP Definition of Done

- [ ] All 13 "Must Have" features implemented, tested, and deployed
- [ ] API responds correctly to a validation suite of 500+ mathematical test cases covering all MVP functions
- [ ] Edge cases handled gracefully: division by zero, overflow, underflow, invalid expressions, domain errors (e.g., log(-1))
- [ ] API uptime demonstrated at 99.9% over a 2-week burn-in period before public launch
- [ ] p50 response time under 50ms, p99 under 200ms measured over burn-in period
- [ ] Documentation portal live with endpoint reference, code examples (Python, JavaScript, cURL), and interactive sandbox
- [ ] Billing integration functional: free tier enforced, paid tier purchasable, usage tracked accurately
- [ ] OpenAPI 3.x specification published and downloadable
- [ ] Security review completed: API key authentication, rate limiting, input validation, no injection vulnerabilities
- [ ] Load test completed: API handles 1,000 concurrent requests without degradation
- [ ] Stakeholder demo completed and sign-off received

---

# Risks and Dependencies

## Key Risks

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Low developer adoption: market prefers local libraries over hosted APIs for math | Medium | High | Free tier lowers barrier. Emphasize cross-platform consistency and expression evaluation as differentiators that local libraries lack. Monitor signup-to-active-use conversion. |
| Accuracy bugs damage credibility: a wrong calculation result reported publicly | Low | Critical | Comprehensive test suite (500+ cases per function), comparison against reference implementations (Wolfram Alpha, Python mpmath), automated regression testing on every deploy. |
| Expression parser edge cases: unexpected input causes crashes or wrong results | Medium | High | Fuzz testing with randomized expressions, explicit grammar definition, sandbox the parser to prevent injection. |
| Free tier abuse: bots or scrapers consume resources without converting | Medium | Medium | Rate limiting per API key, CAPTCHA on signup, anomaly detection on usage patterns. Adjust free tier limit if needed. |
| Stripe billing integration delays MVP launch | Low | Medium | Begin billing integration in month 2. Use manual invoicing as temporary fallback if needed. |
| Single-region outage takes down the entire service | Low | High | Deploy to two availability zones within US-East-1. Multi-region is Phase 2 but AZ redundancy provides baseline resilience. |

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Competitor launches similar API during our development | Low | Medium | Speed to market is the primary defense. 6-month MVP timeline. Focus on developer experience as a moat: documentation quality, response time, error messages. |

## External Dependencies

- **Stripe** - Payment processing and subscription management - Available, well-documented API
- **Auth0 or AWS Cognito** - Developer authentication - Available, evaluate during month 1
- **Domain registrar** - calcengine.io domain (or similar) - Must secure before documentation site goes live
- **Cloud provider (AWS)** - Compute, API Gateway, database - Available, no approval needed
- **SSL certificate provider** - TLS for API and documentation site - Available via AWS Certificate Manager

## Open Questions

- [ ] Should the expression evaluator support variable assignment (e.g., "x = 5; 2*x + 3") or only single-expression evaluation in the MVP?
- [ ] What is the maximum expression length the parser should accept? 1KB? 10KB? Need to balance flexibility against abuse potential.
- [ ] Should the API return results as strings (preserving precision representation) or as JSON numbers (risking floating-point serialization issues)?
- [ ] Do we need to support implicit multiplication (e.g., "2pi" meaning "2 * pi") or require explicit operators?
- [ ] Should the free tier require a credit card on file to reduce abuse, or is email-only signup better for adoption?
- [ ] What is the cancellation and refund policy for paid subscriptions?
- [ ] Should we publish accuracy benchmarks comparing CalcEngine results against Wolfram Alpha and Python mpmath on the documentation site?